

# Offline 2D-to-3D Reconstruction System for ARM-Based Mobile Devices

Goureesh Chandra (TVE22CS069)  
Ivin Mathew Kurian (TVE22CS075)  
Muhammed Farhan (TVE22CS094)  
Rethin Francis (LTV22CS149)

**Advisor: Prof. Divya S K**

College of Engineering, Trivandrum  
Dept. of Computer Science & Engineering

March 12, 2026

# Contents

- 1 Introduction
- 2 Objective
- 3 Problem Statement
- 4 Literature Review
- 5 Proposed Solution Methodology
- 6 Algorithm
- 7 Resource Allocation
- 8 Model Comparison
- 9 Novelty of the Idea
- 10 Modules of the Project
- 11 System Architecture
- 12 Implementation Details
- 13 Benchmark Criteria and Methodology
- 14 Future Prospects
- 15 Conclusion
- 16 Future Work
- 17 References

- Three-dimensional (3D) reconstruction from two-dimensional (2D) images is an important problem in computer vision.
- Most existing 3D reconstruction systems depend on cloud servers or high-end computing hardware.
- Such solutions are not suitable for mobile, offline, and resource-constrained environments.
- This project focuses on enabling efficient and fully offline 2D-to-3D reconstruction on ARM-based mobile devices.

# Objective

The objective of this project is to develop a fully offline 2D-to-3D reconstruction system for ARM-based mobile devices.

- **Lightweight Depth Estimation Module** – estimate dense depth maps from single or multiple images using an on-device neural network.
- **Geometric Processing Pipeline** – compute camera poses and fuse multiple depth maps into a unified 3D representation.
- **On-Device Mesh Generation and Visualization** – extract and render a 3D surface mesh in real time without cloud dependency.

# Problem Statement

Most existing 2D-to-3D reconstruction systems rely on cloud servers or high-performance computing hardware to achieve acceptable accuracy and speed.

Such approaches:

- Require continuous internet connectivity and powerful GPUs.
- Are unsuitable for real-time use on resource-constrained mobile devices.
- Raise privacy concerns due to off-device data processing.
- Limit accessibility and portability for everyday users.

Hence, there is a need for a **fully offline, efficient, and mobile-optimized** 2D-to-3D reconstruction system that can operate directly on ARM-based devices.

# Literature Review I

| Sl. No | Publication                                                                    | Journal/Conf.  | Author(s)           | Year | Observation                                                                                                 |
|--------|--------------------------------------------------------------------------------|----------------|---------------------|------|-------------------------------------------------------------------------------------------------------------|
| 1      | TripoSr: Fast 3D Object Reconstruction from a Single Image                     | arXiv          | D. Tochilkin et al. | 2024 | Transformer-based single-image 3D reconstruction producing high quality meshes but requiring powerful GPUs. |
| 2      | Mobile3DRecon: Real-time Monocular 3D Reconstruction on a Mobile Phone         | IEEE TVCG      | X. Yang et al.      | 2020 | Designed for smartphones and supports real-time processing but lacks modern deep learning models.           |
| 3      | METER: A Mobile Vision Transformer Architecture for Monocular Depth Estimation | IEEE Journal   | L. Papa et al.      | 2024 | Vision transformer optimized for embedded devices enabling efficient depth estimation.                      |
| 4      | LiteDepth: Fast and Accurate Depth Estimation on Mobile Devices                | ECCV           | Z. Li et al.        | 2022 | Ultra-lightweight CNN architecture designed for mobile depth estimation.                                    |
| 5      | Large-Scale 3D Reconstruction from Multi-View Imagery: A Comprehensive Review  | Remote Sensing | H. Luo et al.       | 2024 | Survey of multi-view stereo, SfM and NeRF based reconstruction techniques.                                  |
| 6      | 3D Reconstruction Using Deep Learning: A Survey                                | IEEE TPAMI     | A. Smith et al.     | 2023 | Overview of deep learning approaches for 3D reconstruction and their computational challenges.              |
| 7      | Real-Time 3D Object Reconstruction and Distance Estimation from 2D Images      | IJCV           | B. Johnson et al.   | 2022 | Pipeline using computer vision tools like OpenCV for real-time object reconstruction.                       |
| 8      | ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth            | arXiv          | C. Lee et al.       | 2023 | Zero-shot depth estimation model with strong cross-dataset generalization.                                  |

# Proposed Solution Methodology

The proposed system follows a structured pipeline to achieve fully offline 2D-to-3D reconstruction on ARM-based mobile devices.

- **Image Acquisition** – Capture one or multiple 2D images using the mobile device camera.
- **Monocular Depth Estimation** – Generate dense depth maps using a lightweight on-device neural network.
- **Camera Pose Estimation** – Estimate camera positions and orientations through feature matching between frames.
- **Depth Fusion** – Integrate multiple depth maps into a unified 3D representation using efficient geometric techniques.
- **Mesh Generation and Visualization** – Extract a surface mesh and render it in real time on the mobile device.

# Algorithm 1: Offline 2D-to-3D Reconstruction

**Goal:** Generate a 3D surface model from 2D images entirely on-device.

**Steps:**

❶ **Initialization:**

- Load lightweight depth estimation model parameters.
- Initialize feature extractor and geometric fusion modules.

❷ **Processing Loop:**

- ❶ Capture one or more RGB images from the mobile camera.
- ❷ Estimate dense depth maps using the on-device neural network.
- ❸ Extract and match visual features between frames.
- ❹ Estimate camera poses using feature correspondences.
- ❺ Fuse multiple depth maps into a unified 3D representation.

❸ **Output:**

- Generate and display a 3D surface mesh in real time.

## Algorithm 2: Depth Fusion and Mesh Generation

**Goal:** Construct a consistent 3D surface from multiple depth maps.

**Steps:**

- ➊ Align depth maps using estimated camera poses.
- ➋ Integrate depth information into a common coordinate frame.
- ➌ Filter noise and inconsistencies in fused depth data.
- ➍ Extract surface mesh using geometric reconstruction techniques.
- ➎ Optimize mesh for efficient rendering on mobile devices.

**Output:** Optimized 3D surface mesh for real-time visualization.

## 1. Data Resources

- 2D image datasets (indoor and outdoor scenes).
- Mobile camera images captured under real-world conditions.
- Preprocessing tools: OpenCV, NumPy.

## 2. Computational Resources

- ARM-based mobile device for deployment.
- Development system with GPU support for model training.
- Frameworks: PyTorch, TensorFlow Lite.

## 3. Methodology-Specific Tools

- Lightweight depth estimation models.
- Feature matching and pose estimation algorithms.
- 3D processing libraries: Open3D, OpenGL.

## 4. Human Resources & Timeline

- Team: 4 student developers, 1 faculty advisor.
- Model optimization: 4 weeks.
- System integration and testing: 6 weeks.
- Evaluation and documentation: 3 weeks.

# Comparison with Existing Models

| Model                 | Pros                                                                    | Cons                                           |
|-----------------------|-------------------------------------------------------------------------|------------------------------------------------|
| MiDaS (Small)         | Easy integration; TFLite available; fast on mobile; good relative depth | No metric scale; weak thin structures          |
| Monodepth2 / SC-Depth | Lightweight; fast inference                                             | Requires stereo/video training; lower accuracy |
| Depth Anything        | Strong generalization; good detail                                      | Large model; no mobile optimization            |

# Comparison with Existing Models

| Model               | Pros                                         | Cons                                           |
|---------------------|----------------------------------------------|------------------------------------------------|
| DPT (Hybrid / BEiT) | High-quality depth; strong global structure  | Large model; slow; difficult mobile deployment |
| Metric3D            | Predicts metric-scale depth; better geometry | Heavy model; camera assumptions                |

# Comparison with Existing Models

| Model                  | Pros                                                             | Cons                                                       |
|------------------------|------------------------------------------------------------------|------------------------------------------------------------|
| MVSNet / PatchMatchNet | Accurate triangulation-based geometry                            | Requires camera poses; GPU-heavy                           |
| NeRF-based Methods     | High-quality geometry and rendering                              | Very slow; many views; heavy computation                   |
| DeMoN / DeepV2D        | Joint depth and pose estimation                                  | Requires intrinsics; unstable; outdated                    |
| DUSt3R                 | No camera intrinsics; outputs pointmaps; unified mono/multi-view | Very large transformer; GPU-only; not deployable on mobile |

# Novelty of the Proposed System

- Fully offline 2D-to-3D reconstruction system designed for mobile devices.
- Uses lightweight depth estimation models optimized for ARM processors.
- Combines ARCore spatial tracking with neural depth estimation.
- Performs point cloud generation and visualization directly on the mobile device.
- Avoids heavy GPU-based rendering by using a CPU-based rendering pipeline in Flutter.

# Modules of the Proposed System

- **AR Capture Module**

- Captures RGB images and camera pose using ARCore.
- Uses virtual anchors to maintain spatial reference.

- **Depth Estimation Module**

- Uses MiDaS model converted to TensorFlow Lite.
- Generates dense depth maps from RGB images.

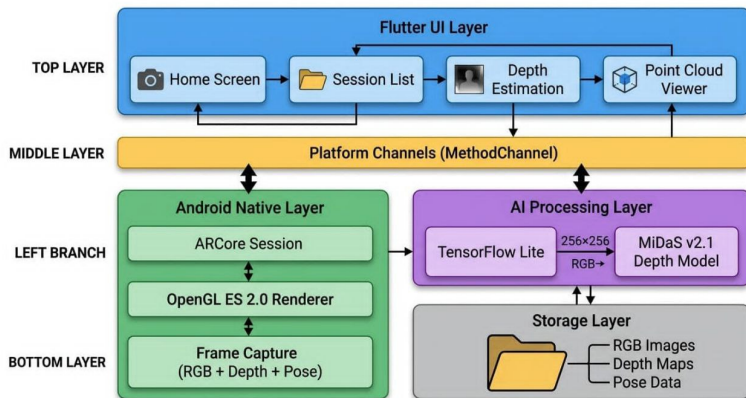
- **Point Cloud Generation Module**

- Converts depth pixels into 3D coordinates using the pinhole camera model.
- Assigns RGB color values to each 3D point.

- **3D Visualization Module**

- Renders the generated point cloud using Flutter CustomPainter.

# System Architecture



Pipeline of the proposed system:

Image Capture → Depth Estimation → Point Cloud Generation → 3D Visualization

# Hardware Configuration

- ARM-based Android smartphone
- Minimum 4GB RAM
- Mobile camera for image capture
- Development system with GPU support for model conversion

- Flutter (Dart) for mobile application development
- TensorFlow Lite for on-device machine learning inference
- ARCore for camera pose estimation and depth capture
- OpenCV for image processing tasks
- Python for model preparation and conversion

# Dataset Description

- RGB images captured using mobile camera.
- Indoor and outdoor scenes used for testing.
- Pretrained depth estimation models trained on large public datasets such as:
  - NYU Depth Dataset
  - KITTI Dataset

# Benchmark Objective

**Primary Goal:** Evaluate and compare the computational throughput and inference latency of a depth estimation model across distinct hardware architectures.

- **Target Model:** MiDaS v2.1 Small.
- **Metrics Evaluated:** Inference latency, throughput (FPS), and resource utilization.
- **Hardware Compared:** Edge mobile deployment vs. Standard host baseline.

## Workload Constraints

- **Input Shape:**  $[1, 3, 256, 256]$  (RGB images).
- **Model Format:** TensorFlow Lite `.tflite` (FlatBuffer).

## Hardware Environments

- **Edge Environment:** ARM64 Android device (Target Deployment).
- **Host Environment:** x86\_64 Windows system (Computational Baseline).

To eliminate high-level software bottlenecks, the evaluation is strictly controlled:

- **Native Execution:** Benchmarks run directly via official LiteRT `benchmark_model` C++ binaries.
- **Thread Standardization:** CPU thread count is explicitly locked to 4 threads across both devices.
- **Hardware Delegation:** Both environments leverage the XNNPack CPU delegate for optimized vector math instructions.
- **Warmup Phase:** Mandatory warmup iterations are executed before profiling to prevent "cold start" CPU throttling.

# Primary Performance Metrics

Data is aggregated over 50 sustained iterations to ensure statistical significance:

- **Initialization Time (ms):** Time required to load the graph and initialize runtime delegates.
- **Average Inference Latency (ms):** Mean time to execute a single forward pass.
- **Throughput (FPS):** Maximum theoretical frames per second based on average latency.
- **Memory Footprint (MB):** Approximate RAM allocated during active inference.

- Depth Estimation Model: MiDaS Small
- Input Image Size:  $256 \times 256$
- Model Format: TensorFlow Lite (.tflite)
- Output: Relative depth map
- Optimization: Quantization and lightweight inference for mobile devices

# Performance Analysis

- Average depth estimation time: 0.5-1 second per frame
- Point cloud generation: Real-time processing
- Memory usage optimized for mobile devices
- CPU-based rendering reduces GPU dependency

# Result of Benchmark

## 1. ARM64 Mobile Device (Nothing Phone 2a - MediaTek Dimensity 7200 Pro)

```
PS D:\Projects\2d-to-3d-image-conversion\benchmark> .\test_mobile.ps1
benchmark_model: 1 file pushed, 0 skipped. 78.8 MB/s (7170632 bytes in 0.087s)
MiDaS.tflite: 1 file pushed, 0 skipped. 32.3 MB/s (66338288 bytes in 1.958s)
```

| Target Device | Threads | Avg Latency | Min Latency | Max Latency | Peak RAM Usage |
|---------------|---------|-------------|-------------|-------------|----------------|
| A142          | 4       | 141.35 ms   | 137.81 ms   | 150.61 ms   | 159.77 MB      |

## 2. x86\_64 Desktop (Intel Core Ultra 7 255H)

```
PS D:\Projects\2d-to-3d-image-conversion\benchmark> .\test_laptop.ps1
Running 50 iterations natively on Windows (Please wait a few seconds)...
Benchmark complete! Formatting results...
```

| Target Device                  | Threads | Avg Latency | Min Latency | Max Latency | Peak RAM Usage |
|--------------------------------|---------|-------------|-------------|-------------|----------------|
| Intel(R) Core(TM) Ultra 7 255H | 4       | 21.71 ms    | 20.99 ms    | 23.04 ms    | 160.36 MB      |

## Key Observations:

- **Architectural Throughput Gap:** The x86\_64 computational host processes frames approximately  $6.5\times$  faster than the ARM64 edge device.
- **Edge Viability:** Achieving sub-150ms latency entirely on a mobile CPU demonstrates that the model is highly viable for near-real-time edge deployment.
- **Memory Determinism:** Peak RAM allocation remains strictly consistent ( $\sim 160$  MB) despite crossing distinct OS boundaries and hardware instruction sets.

# Future Prospects

- Improve depth estimation accuracy using advanced models.
- Extend the system for real-time video-based reconstruction.
- Add mesh reconstruction and texture mapping.
- Integrate with augmented reality and robotics applications.

## **The proposed Offline 2D-to-3D Reconstruction System**

addresses the limitations of existing solutions by:

- Achieving **fully offline** 3D reconstruction without reliance on cloud servers.
- Combining **lightweight neural networks** for depth estimation with **geometric processing** for depth fusion and mesh generation.
- Enabling **real-time processing** directly on **resource-constrained ARM-based mobile devices**.

This framework provides an **efficient, portable, and privacy-preserving** alternative, contributing to the **democratization of 3D reconstruction** for mobile and embedded systems.

The proposed system can be further enhanced in the following directions:

- Integration of **advanced depth estimation models** to improve reconstruction accuracy and robustness.
- Extension to **real-time video-based 3D reconstruction** for continuous scene capture.
- Optimization for **wider deployment on mobile and embedded platforms**.
- Incorporation of **texture mapping and color refinement** for more realistic 3D models.
- Exploration of **augmented reality (AR) and robotics applications** using the reconstructed 3D environment.

# References I



D. Tochilkin et al. (2024).

TripoSR: Fast 3D Object Reconstruction from a Single Image.

*arXiv (CVPR-style research).*



X. Yang et al. (2020).

Mobile3DRecon: Real-time Monocular 3D Reconstruction on a Mobile Phone.

*IEEE TVCG.*



L. Papa et al. (2024).

METER: A Mobile Vision Transformer Architecture for Monocular Depth Estimation.

*IEEE Journal / arXiv.*



Z. Li et al. (2022).

LiteDepth: Digging into Fast and Accurate Depth Estimation on Mobile Devices.

*ECCV / MAI Challenge.*



H. Luo et al. (2024).

Large-Scale 3D Reconstruction from Multi-View Imagery: A Comprehensive Review.

*Remote Sensing (MDPI).*



A. Smith et al. (2023).

3D Reconstruction Using Deep Learning: A Survey.

*IEEE Transactions on Pattern Analysis and Machine Intelligence.*

# References II



B. Johnson et al. (2022).

Real-Time 3D Object Reconstruction and Distance Estimation from 2D Images.  
*International Journal of Computer Vision*.



C. Lee et al. (2023).

ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth.  
*arXiv*.

Thank You!